

Програмні засоби наукових обчислень

Лабораторна робота №1

Варіант 3

Студентки 2-го курсу
Спеціальності ПМ
Гирба М.С

Зміст

<u>Завдання №1.....</u>	<u>3</u>
Завдання №2.....	5
Завдання №3.....	7
Завдання №4.....	10

Завдання №1

Условие: Вычислить число e с заданной точностью ε , используя второй замечательный предел

$$\lim_{x \rightarrow 0} (1 + x)^{1/x} = e$$

Данный предел можно вычислить, уменьшая x к нулю, например, уменьшая в 2 раза на каждом шаге. В таком случае точность считается достигнутой, если на некотором шаге выполнено неравенство $|l(x) - l(x/2)| < \varepsilon$, где $l(x) = (1 + x)^{1/x}$. В этом случае e можно положить равным $l(x/2)$.

Код решения:

```
import sys

sys.stdout.reconfigure(encoding='utf-8')

def calculate_e(epsilon):
    x = 1
    e = 0
    while True:
        new_e = (1 + x) ** (1 / x)
        if abs(new_e - e) < epsilon:
            return new_e
        e = new_e
        x /= 2

epsilon = float(input("epsilon = "))
result = calculate_e(epsilon)
print("Приближенное значение e с точностью", epsilon, "равно:", result)
```

Результат тестов:

```
epsilon = 0.00001  
Приближенное значение e с точностью 1e-05 равно: 2.718276643766046
```

```
epsilon = 0.1  
Приближенное значение e с точностью 0.1 равно: 2.6379284973666
```

```
epsilon = 0.001  
Приближенное значение e с точностью 0.001 равно: 2.7176184823368796
```

Завдання №2

Условие: Запишите свой номер телефона (без учёта ведущего нуля) в виде 9-значного целого числа.

Найдите первую с конца нечётную цифру числа. «Зачеркните» все цифры, следующие за найденной. Верно ли, что оставшееся число – простое?

Пример: 01.09.2020 → 01092020 → 0109=109.

Код решения:

```
import sys

sys.stdout.reconfigure(encoding='utf-8')

def is_prime(num):
    """Проверяет, является ли число простым"""
    if num <= 1:
        return False
    if num <= 3:
        return True
    if num % 2 == 0 or num % 3 == 0:
        return False
    i = 5
    while i * i <= num:
        if num % i == 0 or num % (i + 2) == 0:
            return False
        i += 6
    return True

def check_number(number):
    """Проверяет число по условию задачи"""
    number = str(number)
    for i in range(len(number) - 1, -1, -1):
        if int(number[i]) % 2 != 0:
            new_number = int(number[:i+1])
            if is_prime(new_number):
```

```
        print(f"Число {new_number} - простое")
    else:
        print(f"Число {new_number} - составное")
    return
print("В числе нет нечетных цифр")

phone_number = int(input("Введите номер телефона " ))
check_number(phone_number)
```

Результат тестов:

```
Введите номер телефона 688014884
Число 68801 - составное
```

```
Введите номер телефона 1096666666
Число 109 - простое
```

```
Введите номер телефона 648462464
В числе нет нечетных цифр
```

Завдання №3

Условие: Концерт, посвящённый Дню математика, - довольно популярное мероприятие у студентов. Однако не все желающие смогут на него попасть. В зрительном зале всего 15 рядов, в каждом из которых 30 мест. Информация о проданных билетах хранится в матрице, номера строк которой соответствуют номерам рядов, в номера столбцов – номерам мест. Если билет на то или иное место продан, то соответствующий элемент матрицы равен 1, если же место свободно – 0. Весёлая кампания первокурсников, поздно узнавших о концерте, хочет найти ряд, в котором осталось наибольшее число свободных мест, идущих подряд. Помогите им.

Код решения:

```
import random

def find_best_row(matrix, min_free_seats=1):
    max_free_seats = 0
    max_free_row = -1

    for i in range(len(matrix)):
        current_free_seats = 0
        for j in range(len(matrix[0])):
            if matrix[i][j] == 0:
                current_free_seats += 1
            else:
                if current_free_seats >= min_free_seats and
current_free_seats > max_free_seats:
                    max_free_seats = current_free_seats
                    max_free_row = i
                    current_free_seats = 0

        if current_free_seats >= min_free_seats and current_free_seats >
max_free_seats:
            max_free_seats = current_free_seats
```

```

        max_free_row = i

    return max_free_row + 1 # Номера рядов начинаются с 1
def generate_random_matrix(rows, cols, min_value, max_value):
    matrix = [[random.randint(min_value, max_value) for _ in range(cols)]
for _ in range(rows)]
    return matrix

def print_matrix(matrix):

    for row in matrix:
        print(row)

rows = 15
cols = 30
min_value = 0
max_value = 1

matrix = generate_random_matrix(rows, cols, min_value, max_value)

print("Сгенерированная матрица:")
print_matrix(matrix)

best_row = find_best_row(matrix, min_free_seats=3)
print("Лучший ряд для компании:", best_row)

```

Результат тестов:

```

[0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 1, 0, 0, 0, 1, 0, 1, 0, 0, 0, 1, 1, 1, 0, 0, 0, 1, 1, 1, 0]
[1, 0, 0, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1, 0, 0, 1, 1, 1, 0, 1, 1, 0, 1, 1, 1, 0, 0, 1, 1, 1]
[0, 0, 0, 0, 1, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0, 0, 1, 0, 0, 0]
[0, 0, 0, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0]
[1, 1, 1, 1, 1, 1, 1, 0, 0, 1, 0, 1, 1, 0, 1, 1, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0]
[0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 1, 0, 0]
[1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1, 1, 0, 1, 0, 1, 0, 1, 1, 0]
[0, 0, 0, 1, 0, 1, 1, 1, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 0, 1, 1, 0, 0, 0, 1, 1, 1, 1]
[0, 0, 1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 0, 1, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 1, 1, 0, 1, 1, 0]
[0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1]
[1, 1, 1, 0, 1, 0, 0, 1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1]
[0, 1, 1, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 1, 0, 1, 0, 0, 0, 1, 1]
[1, 1, 1, 1, 0, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 1, 1, 0, 1, 0, 1, 0, 0]
[0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 0]
[0, 0, 0, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1]
Лучший ряд для компании: 15

```



```
[1, 1, 0, 0, 1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 1, 1]
[1, 1, 1, 0, 1, 0, 0, 0, 0, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 0, 0]
[0, 0, 1, 1, 0, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 1, 0, 1, 0, 1, 0, 0]
[0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 0, 1, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1]
[0, 1, 1, 0, 1, 0, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0]
[0, 0, 1, 1, 0, 0, 0, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 1, 0, 0, 0, 0, 1, 0, 1, 1]
[1, 0, 0, 0, 1, 1, 1, 1, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 1]
[1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 0, 0, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 1, 1, 0, 0, 0]
[0, 1, 1, 0, 0, 0, 0, 0, 1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1, 0, 0, 0, 1, 0, 1, 0, 0]
[1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1]
[0, 1, 1, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1, 0, 0, 0, 1]
[1, 1, 1, 0, 0, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0]
[1, 1, 1, 1, 0, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 1, 1, 1, 0, 1, 0, 0, 1, 0]
[1, 0, 1, 1, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 1, 0, 1, 0, 1, 0, 0, 0, 1, 1, 1, 1, 0, 1, 0]
[0, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1, 0, 1, 0, 0, 1, 0, 0]
Лучший ряд для компании: 1
```

Завдання №4

Условие: Во входном файле записан некоторый достаточно длинный текст. Найдите процент коротких (менее 4 символов) и длинных (более 10 символов) слов в этом тексте.

Код решения:

```
def count_words_by_length(filename, min_length=4, max_length=10):
    with open(filename, 'r', encoding='utf-8') as file:
        text = file.read()

    words = text.split()
    total_words = len(words)

    short_words = [word for word in words if len(word) < min_length]
    long_words = [word for word in words if len(word) > max_length]

    return total_words, len(short_words), len(long_words)
```

```
filename = "LR1/text1.txt"
min_length = 4
max_length = 10
total, short, long = count_words_by_length(filename, min_length,
max_length)

percent_short = (short / total) * 100
percent_long = (long / total) * 100

print(f"Общее количество слов: {total}")
print(f"Процент коротких слов ({min_length} символов и меньше):
{percent_short:.2f}%")
print(f"Процент длинных слов ({max_length} символов и более):
{percent_long:.2f}%")
```

Результат тестов:

```
Общее количество слов: 63
Процент коротких слов (4 символов и меньше): 28.57%
Процент длинных слов (10 символов и более): 7.94%
```

```
Общее количество слов: 133
Процент коротких слов (4 символов и меньше): 33.08%
Процент длинных слов (10 символов и более): 7.52%
```